

Designing a RAG Pipeline for a Large Scale Enterprise Knowledge Base with Real Time Constraints

1. Overview

A Retrieval Augmented Generation (RAG) system combines information retrieval with large language models to generate accurate and context aware responses. In a large scale enterprise knowledge base, the system must handle large volumes of documents while maintaining low latency and real time response times.

To achieve this, the architecture is typically divided into two main components:

- Indexing Pipeline (Offline Pipeline)
- Generation Pipeline (Online / Real-Time Pipeline)

The indexing pipeline prepares and organizes the knowledge base for efficient retrieval, while the generation pipeline processes user queries and generates answers in real time.

2. Indexing Pipeline (Offline Processing)

The indexing pipeline processes enterprise documents and prepares them for fast retrieval during query time. This pipeline usually runs offline or periodically in batch jobs.

Steps in the Indexing Pipeline

1. Data Ingestion

Enterprise documents are collected from multiple sources such as:

- Internal knowledge bases
- PDFs and reports
- Databases
- Emails
- APIs
- Logs and support tickets
- Wiki pages

These documents are converted into a standardized text format.

2. Data Cleaning and Normalization

The collected documents are cleaned to remove noise and irrelevant formatting. Important metadata is extracted, such as:

- Document source
- Author
- Department
- Timestamp
- Document type

This metadata helps improve retrieval accuracy.

3. Document Chunking

Large documents are divided into smaller semantic chunks.

Typical configuration:

- Chunk size: 200 to 500 tokens
- Overlap: 20 to 50 tokens

Chunking is important because LLMs and vector search work better with smaller, focused text segments.

4. Embedding Generation

Each chunk is converted into a vector embedding using an embedding model such as:

- BGE
- E5
- OpenAI embeddings
- Sentence Transformers

These embeddings capture the semantic meaning of the text.

5. Vector Index Construction

The embeddings are stored in a vector database to enable fast similarity search.

Common vector databases include:

- FAISS
- Pinecone
- Milvus
- Weaviate

These systems allow fast retrieval even when the dataset contains millions or billions of documents.

6. Keyword Indexing

In addition to vector search, a keyword based search index is created using systems like:

- Elasticsearch
- BM25

This helps retrieve documents containing exact terms such as:

- product codes
- error IDs
- technical terms

7. Metadata Enrichment

Each chunk is stored along with additional metadata such as:

- document title
- tags
- timestamps
- access permissions

This enables filtered search and enterprise security compliance.

8. Index Storage and Scaling

To support large enterprise datasets, indexes are distributed across multiple servers using:

- sharding
- distributed storage
- approximate nearest neighbor (ANN) search

This ensures fast retrieval even with billions of documents.

3. Generation Pipeline (Real-Time Query Processing)

The generation pipeline runs when a user submits a query. It must operate under strict latency constraints to ensure fast responses.

Steps in the Generation Pipeline

1. User Query Input

A user submits a query through an interface such as:

- chatbot
- search system
- enterprise assistant

2. Query Processing

The system processes the query by:

- cleaning the text
- converting the query into a vector embedding using the same embedding model used in indexing.

3. Hybrid Retrieval

Relevant document chunks are retrieved using a hybrid retrieval approach, which combines:

- vector similarity search
- keyword search

This improves retrieval accuracy.

4. Candidate Selection

The system retrieves the top-k relevant chunks based on similarity scores and metadata filters.

Example:

Top 10–20 chunks are selected as candidate documents.

5. Re-Ranking

A re-ranking model (such as a cross-encoder) evaluates the relevance of retrieved documents with respect to the query.

The most relevant top-5 chunks are selected for the final context.

6. Context Assembly

The selected chunks are combined to form the context that will be provided to the LLM.

This context ensures the LLM generates accurate answers based on enterprise knowledge.

7. Prompt Construction

A structured prompt is created containing:

- the user question
- retrieved document context
- instructions for the model

Example structure:

Question + Retrieved Context + Instructions

8. LLM Generation

A large language model generates the final answer using the retrieved context.

Examples of LLMs:

- Llama
- GPT
- Claude

9. Response Delivery

The generated answer is returned to the user.

Often, the response also includes citations or references to the retrieved documents.

4. Key Design Considerations for Real-Time RAG Systems

To maintain real-time performance in enterprise environments, the following strategies are used:

- Use Approximate Nearest Neighbor (ANN) search to reduce vector search latency.
- Implement hybrid retrieval (vector + keyword) for higher accuracy.
- Cache frequently asked queries and results.
- Limit the number of retrieved chunks to control LLM context size.
- Use scalable vector databases with horizontal sharding.
- Run indexing pipelines asynchronously to continuously update knowledge bases.